UNIJORGE

ENGENHARIA DA COMPUTAÇÃO

GRUPO 2

CHIPLAB

Sistema educacional para explorar
arquitetura de processadores

Alexandre

Eliel

Leonardo

Luan

Maitê

Marcos

Whanderlon

Willian

Salvador

2026

Alexandre
Eliel
Leonardo
Luan
Maitê
Marcos
Whanderlon
Willian

CHIPLAB

Sistema educacional para explorar
arquitetura de processadores

Trabalho Integrador apresentado à disciplina de Qualidade de Produto de Software do curso de Engenharia da Computação da Unijorge, como requisito parcial da avaliação AV3. Primeira etapa. Professora responsável: Cristina Weber Cal.

Salvador
2026

Resumo

O trabalho apresenta o ChipLab, um sistema desktop educacional para explorar uma arquitetura de processador por meio da montagem e da execução de programas em um conjunto de instruções próprio. O projeto atende à primeira etapa da atividade integradora de Qualidade de Produto de Software do curso de Engenharia da Computação da Unijorge. A metodologia seleciona uma demanda pública sobre a observação simultânea de código e memória e compara sua viabilidade com uma solicitação de conversão de Verilog para circuitos. A solução delimita o escopo a uma CPU simulada, com registradores, memória, desvios, validações e exportação de uma unidade lógica e aritmética combinacional. A especificação organiza requisitos funcionais, requisitos de qualidade e regras de negócio, relacionando suas origens a componentes e critérios objetivos de aceitação. O material prepara a transferência do código e da documentação ao Grupo 1 para avaliação independente. A verificação interna registra 104 verificações do núcleo e 18 de integração aprovadas, além da soma com resultado 15 em 20 ciclos. A etapa atual entrega a aplicação e uma base verificável para os testes; cobertura estrutural, escore de mutação e resultados de auditoria permanecem reservados à segunda etapa. A proposta não representa uma ferramenta de fabricação de chips nem apresenta certificação normativa.

Palavras-chave: arquitetura de processadores; simulação educacional; requisitos de software; qualidade de software; aplicação desktop.

Abstract

This work presents ChipLab, an educational desktop system for exploring processor architecture through the assembly and execution of programs written in a custom instruction set. The project addresses the first stage of the Software Product Quality course assignment in the Computer Engineering program at Unijorge. Its method selects a public request for viewing source code and memory simultaneously and compares its feasibility with a request to convert Verilog descriptions into circuits. The solution limits its scope to a simulated CPU with registers, memory, conditional branches, input validation, and the export of a combinational arithmetic and logic unit. The specification organizes functional requirements, quality requirements, and business rules, linking their origins to software components and objective acceptance criteria. The package prepares the transfer of source code and documentation to Group 1 for independent assessment. Internal verification records 104 passed core checks and 18 passed integration checks, and the sample summation produces 15 in 20 cycles. The current stage delivers the application and a verifiable basis for subsequent tests; structural coverage, mutation scores, and external audit results are reserved for the second stage. The proposal is an educational software artifact rather than a chip manufacturing tool and does not claim formal standards certification.

Keywords: processor architecture; educational simulation; software requirements; software quality; desktop application.

Sumário

Introdução 5

Levantamento de fontes e escolha do problema 6

Contraprova de viabilidade e decisão de escopo 7

Delimitação e transformação das necessidades 8

1 Introdução da SRS 9

2 Descrição geral do sistema 10

3 Requisitos específicos e regras de negócio 11

3.1 Requisitos funcionais 11

3.2 Requisitos não funcionais de qualidade 16

3.3 Regras de negócio 18

4 Critérios de verificação e validação 20

5 Matriz de rastreabilidade 22

6 Projeto e implementação 23

7 Organização e transferência 24

8 Considerações finais 25

Referências 26

Referências públicas e anexos 27

Introdução

O ChipLab permite manipular parâmetros de uma arquitetura educacional e observar a execução de instruções em uma interface desktop simples. A escolha responde ao interesse do grupo pela área de hardware e cria um sistema com bifurcações, processamento de dados e validações restritivas, adequado à avaliação de qualidade de software prevista na AV3.

O objetivo geral é entregar, em 27 de outubro de 2026, um programa funcional e uma especificação que um grupo auditor possa usar de forma independente. Os objetivos específicos são tornar visíveis registradores e memória, validar programas antes da execução, controlar a quantidade de passos executados, persistir projetos e exportar uma ALU em Verilog.

A primeira etapa contém levantamento, comparação de duas fontes reais, seleção justificada, requisitos, implementação e documentação. A segunda etapa, com entrega em 24 de novembro de 2026, tratará da auditoria, dos testes estruturais e de mutação e da consolidação de métricas. O sistema deverá permanecer disponível para avaliação até 16 de dezembro de 2026, conforme o enunciado.

Este relatório registra decisões do projeto, critérios de aceitação e organização proposta da equipe. A documentação segue a estrutura solicitada no enunciado para a SRS e utiliza as categorias de qualidade da edição ISO/IEC 25010:2011 indicada pela disciplina. A referência a normas orienta a elaboração e não equivale a uma declaração de certificação.

Os nomes dos integrantes foram fornecidos sem sobrenomes. A identificação é preservada exatamente como recebida e deve ser complementada pela equipe antes da submissão institucional, caso a professora exija nomes completos. Não foram informados centro acadêmico ou departamento; esses dados não foram inventados.

Levantamento de fontes e escolha do problema

Fonte F1 selecionada para orientar o escopo

A solicitação pública de trombonehero, publicada em 2 de junho de 2025 no repositório oficial do Ripes, pede a visualização simultânea de memória e código. O título é Feature request view memory and code disassembly at the same time, issue 408. Trata-se de uma demanda textual de uso em um simulador educacional de processadores, suficientemente concreta para orientar uma funcionalidade observável (Trombonehero, 2025).

https://github.com/mortbopet/Ripes/issues/408

Memory and the disassembly or source code

O trecho acima constitui um recorte literal do texto bruto selecionado, preservado em inglês, com 7 palavras. Os arquivos de fontes registram o recorte exato e seus metadados. O recorte é limitado e não representa o corpo integral da discussão. A interpretação da necessidade, descrita em paráfrase, é observar código e memória em conjunto durante o estudo da execução.

A adaptação ChipLab oferece editor, memória e registradores na mesma janela. A fonte não exige a ISA, as configurações, o formato JSON, o limite de passos ou a geração Verilog aqui definidos. Esses itens são decisões da equipe, identificadas separadamente para evitar atribuição indevida. Também não se alega que o ChipLab seja um fork ou uma nova versão do Ripes.

Critério de triagem

A fonte deve apresentar uma necessidade real, permitir delimitar entradas e saídas, gerar critérios de teste e caber em uma implementação acadêmica. O grupo prioriza a observação de estados e regras determinísticas para facilitar oráculos de teste. As duas fontes são solicitações públicas de funcionalidades; não são contratos com um cliente formal.

Contraprova de viabilidade e decisão de escopo

Fonte F2 mantida como alternativa

A solicitação de quantrpeter, publicada em 31 de dezembro de 2022 no repositório oficial do Logisim evolution, propõe converter descrições Verilog em circuitos Logisim para simulação educacional. O título original é looking for cooperate on verilog converter, issue 1616 (Quantrpeter, 2022).

https://github.com/logisim-evolution/logisim-evolution/issues/1616

> convert verilog into logisim and simulate it

O recorte literal do texto bruto contém 7 palavras e não reproduz o corpo integral da solicitação. A análise de viabilidade é apresentada em paráfrase: a alternativa exigiria definir um subconjunto de Verilog, reconhecer construções, elaborar conectividade, gerar circuitos e validar equivalência. A fonte não especifica versão da linguagem, subconjunto ou tratamento de construções sequenciais. A incompletude aumenta o risco de escopo.

Critério	F1 código e memória	F2 conversor Verilog
Viabilidade técnica	Estados inteiros e simulador sequencial	Parser HDL e tradução de circuitos
Completude do domínio	Necessidade visual explícita; ISA precisa ser delimitada	Subconjunto de HDL e semântica ainda indefinidos
Complexidade dominante	Montagem com rótulos e execução limitada por passos	Análise sintática, elaboração e equivalência
Adequação ao prazo	Maior previsibilidade com escopo reduzido	Risco elevado de entrega parcial
Decisão	Selecionada e adaptada ao ChipLab	Documentada como contraprova

A decisão é implementar a necessidade de F1 em um produto próprio. A exportação da ALU Verilog é uma extensão educacional definida pela equipe; não implementa o conversor solicitado em F2. A comparação é uma análise de viabilidade, sem medição experimental de esforço nem afirmação de impossibilidade técnica da alternativa.

Delimitação e transformação das necessidades

A derivação segue uma cadeia explícita: texto F1 → observação simultânea → RF-007 e RNF-004 → interface principal → CT-007 e CT-015. Os demais requisitos são originados pelas decisões EQUIPE-01 a EQUIPE-12. O enunciado da AV3, identificado como A1, justifica a documentação, a rastreabilidade, a presença de validações e a disponibilização para auditoria.

Origem	Decisão derivada
EQUIPE-01	Arquitetura configurável com valores discretos
EQUIPE-02	Editor e montador de ISA educacional própria
EQUIPE-03	Passo, lote limitado e reinicialização
EQUIPE-04	Estado visível e comandos em português
EQUIPE-05	Projeto JSON versão 1 sem instantâneo
EQUIPE-06	Rastro exportado em CSV
EQUIPE-07	ALU combinacional em Verilog
EQUIPE-08	Oráculo determinístico da soma de 1 a 5
EQUIPE-09	Limite de execução e meta de tempo mensurável
EQUIPE-10	Rejeição sem substituição de estado válido
EQUIPE-11	Entradas restritas sem execução de código hospedeiro
EQUIPE-12	Núcleo testável e dependências padronizadas

O produto não contempla desenho físico do chip, fotolitografia, síntese de uma CPU inteira, caches, previsão de desvios, pipeline ou avaliação de consumo de energia. A configuração representa o modelo lógico utilizado pelo simulador. O único artefato HDL produzido é uma ALU combinacional; sua simulação ou síntese em ferramentas externas exige validação própria.

1 Introdução da SRS

1.1 Propósito do documento

Especificar o comportamento observável do ChipLab versão 1 e permitir implementação, aceitação e auditoria independente. O documento destina-se ao Grupo 2, ao Grupo 1 auditor e à professora Cristina Weber Cal.

1.2 Escopo do produto de software

Aplicação desktop local para configurar uma CPU educacional, montar programas, simular instruções e observar estado. Inclui projetos JSON, rastro CSV e exportação de ALU Verilog. A ISA é própria e não deve ser confundida com RISC-V. As exclusões constam da delimitação e de RN-010.

1.3 Definições siglas e acrônimos

CPU: unidade central de processamento. ALU ou ULA: unidade lógica e aritmética. ISA: conjunto de instruções. PC: índice da próxima instrução. HDL: linguagem de descrição de hardware. RF: requisito funcional. RNF: requisito não funcional. RN: regra de negócio. CT: caso de teste. SRS: especificação de requisitos de software. MiB: 1048576 bytes.

1.4 Referências normativas e técnicas

A1 é o enunciado Av3 Trabalho integrador, páginas 3 a 8. A estrutura segue a organização solicitada de ISO/IEC/IEEE 29148:2018 e usa as oito características de qualidade da ISO/IEC 25010:2011 citada no enunciado. As fontes técnicas F1 e F2 e as normas de apresentação estão nas referências.

1.5 Visão geral da organização do documento

A seção 2 define contexto e restrições. A seção 3 cataloga RF, RNF e RN. A seção 4 define verificação e aceitação. A seção 5 estabelece a relação bidirecional entre origem, requisito, implementação e teste. O arquivo SRS.md reproduz essa estrutura para consulta junto ao código.

2 Descrição geral do sistema

2.1 Perspectiva e contexto do produto

O ChipLab é uma aplicação independente que usa arquivos locais. O usuário fornece configuração e programa; o montador produz instruções validadas; a CPU simulada transforma o estado; a interface apresenta os valores. Não há servidor, conta de usuário ou serviço externo em operação.

2.2 Funções e macrofuncionalidades principais

Configurar arquitetura, editar e montar fonte, executar passo ou lote, reiniciar CPU, observar código e memória, abrir e salvar projetos, exportar rastro e gerar ALU Verilog. As entradas são sempre validadas antes de alterar o estado válido.

2.3 Perfis classes e características dos usuários

Estudante de Engenharia da Computação com conhecimentos iniciais de arquitetura e programação; docente que demonstra instruções; auditor que reproduz comportamentos e inspeciona código. Não há separação de privilégios, pois a aplicação é local e monousuário.

2.4 Ambiente operacional e tecnológico de execução

C++17 com biblioteca padrão e interface nativa da API Win32. A distribuição inclui ChipLab.exe para Windows 64 bits. O núcleo e seu executável de testes independem da janela. O ambiente efetivamente utilizado consta em evidencias. A execução cotidiana é offline e não exige compilador.

2.5 Restrições de projeto linguagem e implementação

Interface Win32 e núcleo C++17 sem frameworks externos. A fonte tem até 100000 bytes UTF-8 e 1024 instruções. Projetos têm até 1 MiB. O lote aceita de 1 a 1000 passos. O nome da arquitetura tem de 1 a 60 pontos Unicode e não pode ser vazio ou somente espaços. A memória tem até 256 palavras.

2.6 Premissas de desenvolvimento e dependências externas

O computador deve executar Windows 64 bits e permitir leitura e gravação dos arquivos escolhidos. A recompilação requer compilador C++17 e os scripts fornecidos. Não se pressupõe simulador ou sintetizador Verilog. Arquivos externos são validados antes de substituir o projeto.

3 Requisitos específicos e regras de negócio

3.1 Requisitos funcionais catalogados e priorizados

RF-001 Configurar a arquitetura

O sistema deve aceitar somente larguras de palavra de 8, 16 ou 32 bits, bancos de 4, 8 ou 16 registradores e memórias de 64, 128 ou 256 palavras. Ao montar com configuração válida, deve iniciar a CPU com PC, ciclos, registradores e memória em zero.

Prioridade: Essencial. Origem: EQUIPE-01. Verificação: CT-001. Componente: src/core.hpp.

Aceitação: Testar as 27 combinações e rejeitar 7 bits, 5 registradores e 63 palavras sem substituir a configuração válida.

RF-002 Editar o programa

O sistema deve fornecer um editor de texto para programas na ISA educacional ChipLab, com fonte limitada pelo núcleo a 100000 bytes em UTF-8.

Prioridade: Essencial. Origem: EQUIPE-02. Verificação: CT-002. Componente: src/main.cpp; src/core.hpp.

Aceitação: Editar, montar e executar o programa de exemplo; rejeitar uma fonte de 100001 bytes.

RF-003 Montar e validar a fonte

O montador deve converter a fonte em até 1024 instruções, resolvendo rótulos e validando opcode, operandos, registradores, imediatos, endereços e saltos. Uma rejeição deve manter o último estado executável válido. Erros ligados a uma instrução devem informar sua linha; erros globais, como fonte vazia ou limite total, podem omiti-la.

Prioridade: Essencial. Origem: EQUIPE-02. Verificação: CT-003. Componente: src/core.hpp.

Aceitação: Aceitar o exemplo; rejeitar rótulo duplicado, alvo inexistente, opcode inválido, registrador fora do banco e programa com 1025 instruções.

3.1 Requisitos funcionais de execução

RF-004 Executar uma instrução

A ação Passo deve executar exatamente uma instrução do programa montado, atualizar o estado segundo a ISA e acrescentar um registro ao rastro de execução.

Prioridade: Essencial. Origem: EQUIPE-03. Verificação: CT-004. Componente: src/core.hpp; src/main.cpp.

Aceitação: Após MOV R0, 5, R0 deve valer 5, o PC deve avançar uma instrução e os ciclos devem crescer em uma unidade.

RF-005 Executar com limite

A ação Executar deve aceitar um limite inteiro entre 1 e 1000 passos e encerrar o lote ao encontrar HALT, um erro ou o limite solicitado.

Prioridade: Essencial. Origem: EQUIPE-03. Verificação: CT-005. Componente: src/core.hpp; src/main.cpp.

Aceitação: Executar JMP 0 com limite 10 e observar retorno após 10 ciclos; rejeitar limites 0 e 1001.

RF-006 Reiniciar a simulação

A ação Reiniciar deve remontar o texto e a configuração atuais e, se forem válidos, zerar PC, ciclos, registradores, memória e rastro e liberar o estado de parada. Se a montagem falhar, o último estado válido deve ser preservado.

Prioridade: Essencial. Origem: EQUIPE-03. Verificação: CT-006. Componente: src/core.hpp; src/main.cpp.

Aceitação: Executar o exemplo, reiniciar sem editar e confirmar valores nulos; repetir a execução e obter novamente 15.

3.1 Requisitos funcionais de observação e projeto

RF-007 Observar o estado

A interface deve exibir o código fonte, o estado dos registradores, a memória, o PC, os ciclos, o estado de execução e o rastro, permitindo observar código e memória na mesma janela.

Prioridade: Essencial. Origem: F1; EQUIPE-04. Verificação: CT-007. Componente: src/main.cpp.

Aceitação: Na janela de demonstração, verificar a presença simultânea do editor e da memória e conferir o valor de R1 e da posição 0 ao final do exemplo.

RF-008 Abrir um projeto

O sistema deve importar um arquivo JSON de projeto ChipLab versão 1, com tamanho máximo de 1 MiB, contendo configuração e fonte, validando integralmente o documento antes de substituir o projeto corrente.

Prioridade: Essencial. Origem: EQUIPE-05. Verificação: CT-008. Componente: src/core.hpp; src/main.cpp.

Aceitação: Abrir projeto válido; rejeitar JSON inválido, versão desconhecida, campos inesperados e arquivo acima do limite, mantendo o projeto anterior.

RF-009 Salvar um projeto

O sistema deve exportar configuração e fonte atuais em JSON de projeto versão 1, sem instantâneo da CPU. Deve aceitar rascunho de sintaxe incompleta, respeitados UTF-8 e limites de tamanho. Abrir o arquivo deve carregar o editor sem executar a fonte; a CPU só é montada por Montar, Passo ou Executar.

Prioridade: Essencial. Origem: EQUIPE-05. Verificação: CT-009. Componente: src/core.hpp; src/main.cpp.

Aceitação: Salvar e reabrir; comparar configuração e fonte, sem assumir restauração da CPU. Montar o exemplo válido e confirmar estado inicial zerado.

3.1 Requisitos funcionais de exportação

RF-010 Exportar o rastro

O sistema deve exportar CSV com os campos ciclo, pc, instrucao, proximo_pc, alteracoes e halt, contendo uma linha por instrução executada. O campo alteracoes deve registrar mudanças de registradores e memória daquela instrução, sem exigir instantâneo completo.

Prioridade: Importante. Origem: EQUIPE-06. Verificação: CT-010. Componente: src/main.cpp.

Aceitação: Executar dois passos e conferir duas linhas de dados no CSV e a correspondência das alterações com a interface.

RF-011 Exportar uma ALU em Verilog

O sistema deve gerar um módulo Verilog combinacional da ALU com largura igual à palavra configurada e operações ADD, SUB, AND, OR e XOR, usando nome de módulo fixo e código sem dependências externas.

Prioridade: Importante. Origem: EQUIPE-07. Verificação: CT-011. Componente: src/core.hpp; src/main.cpp.

Aceitação: Inspecionar a largura de entrada e saída para 8, 16 e 32 bits; conferir as cinco operações e uma saída definida no ramo default.

Semântica do programa de demonstração

O programa soma.asm inicia R0 com 5 e R1 com 0. Cada iteração soma R0 ao acumulador R1 e decrementa R0. JNZ retorna ao laço enquanto R0 for diferente de zero. STORE grava R1 no endereço 0; LOAD lê esse endereço em R2; HALT encerra. O resultado esperado é R0 = 0, R1 = 15, R2 = 15 e memória[0] = 15, com PC = 8 e 20 ciclos.

Esse resultado é um oráculo especificado a priori. Sua presença no relatório não substitui o registro de uma execução. As evidências de testes e as capturas de interface são entregues separadamente, com seus respectivos resultados e limitações.

3.1 Contratos de sintaxe e arquivos

Os operandos são separados por vírgulas. Opcodes e registradores aceitam maiúsculas ou minúsculas; rótulos distinguem maiúsculas e minúsculas e começam por letra ASCII ou sublinhado, seguidos por letras, dígitos ou sublinhado. O rótulo termina com dois-pontos e pode anteceder uma instrução na mesma linha. Comentários começam por # ou ponto e vírgula. Números podem ser decimais ou hexadecimais com prefixo 0x, incluindo sinal.

Forma	Significado
MOV Rd, valor ou Rs	Copia imediato ou registrador para Rd
ADD/SUB/AND/OR/ XOR Rd, Ra, Rb	Aplica a operação a Ra e Rb e grava Rd
ADDI Rd, Rs, imediato	Soma imediato a Rs e grava Rd
LOAD Rd, endereço	Lê a palavra de memória em Rd
STORE Rs, endereço	Grava Rs na palavra de memória
JMP alvo	Desvia incondicionalmente
JZ/JNZ Rs, alvo	Desvia conforme Rs seja zero ou não zero
HALT	Para explicitamente a CPU

O esquema JSON versão 1 exige exatamente três campos na raiz: version (inteiro 1), architecture (objeto) e source (texto). architecture exige exatamente name (texto de 1 a 60 pontos Unicode, sem NUL e não somente espaços), bits (8, 16 ou 32), registers (4, 8 ou 16) e memory_size (64, 128 ou 256). Chaves duplicadas, desconhecidas, ausentes, tipos incorretos, UTF-8 inválido e conteúdo posterior ao documento são rejeitados.

O arquivo tem no máximo 1048576 bytes e source tem no máximo 100000 bytes UTF-8, sem caractere NUL. Um rascunho salvo pode não montar; a validação sintática ocorre na montagem, não na abertura do JSON. A edição nativa normaliza quebras LF ou CRLF para a representação do Windows. Não são armazenados PC, ciclos, registradores, memória ou rastro.

3.2 Requisitos não funcionais de qualidade

As categorias correspondem à ISO/IEC 25010:2011 indicada na atividade. Todas as prioridades são essenciais. A meta de RNF-002 aplica-se ao cenário definido; a medição registrada em evidencias/testes_integracao_win32.txt não garante o mesmo tempo em outros computadores.

RNF-001 Adequação funcional

A implementação deve reproduzir, para o programa soma.asm fornecido, R0 = 0, R1 = 15, R2 = 15, memória[0] = 15, PC = 8 e 20 ciclos após HALT nas configurações suportadas.

Origem: EQUIPE-08. Verificação: CT-012. Método: Teste de software.

RNF-002 Eficiência de desempenho

Um lote deve executar no máximo 1000 instruções por solicitação. O ensaio de aceitação deve registrar o tempo de um lote de 1000 saltos na configuração 8 bits, 8 registradores e 64 palavras, adotando meta de até 2 segundos na máquina identificada no registro.

Origem: EQUIPE-09. Verificação: CT-013. Método: Medição de tempo no cenário definido.

RNF-003 Compatibilidade

O documento JSON exportado deve preservar configuração e conteúdo lógico da fonte ao reabrir no formato versão 1. O núcleo conserva bytes UTF-8 válidos; a interface normaliza quebras de linha para LF ao montar ou salvar e usa CRLF no editor.

Origem: EQUIPE-05. Verificação: CT-014. Método: Teste de ida e volta.

RNF-004 Usabilidade

A janela principal deve disponibilizar as ações de montar, passo, executar e reiniciar por controles rotulados em português; uma falha ligada a uma instrução deve identificar a linha da fonte, ressalvados erros globais.

Origem: F1; EQUIPE-04. Verificação: CT-015. Método: Inspeção e demonstração.

3.2 Requisitos não funcionais de qualidade

RNF-005 Confiabilidade

A rejeição de uma configuração, fonte ou projeto inválido deve preservar o último estado válido, sem execução parcial da entrada rejeitada.

Origem: EQUIPE-10. Verificação: CT-016. Método: Teste com comparação antes e depois.

RNF-006 Segurança

O sistema deve processar arquivos JSON com limite de 1 MiB e comandos restritos à ISA; não deve executar texto da fonte como código nativo nem carregar objetos serializados executáveis.

Origem: EQUIPE-11. Verificação: CT-017. Método: Teste de entradas e inspeção de código.

RNF-007 Manutenibilidade

O núcleo C++17 de montagem, simulação, persistência e geração Verilog deve poder ser compilado em um executável de testes sem abrir a interface e sem frameworks externos.

Origem: EQUIPE-12. Verificação: CT-018. Método: Inspeção e execução de testes.

RNF-008 Portabilidade

A distribuição deve fornecer um executável para Windows 64 bits sem instalação de runtime adicional e o código fonte C++17 para recompilação. A interface Win32 é específica de Windows; o núcleo deve permanecer independente dela. A compatibilidade efetivamente verificada deve ser identificada nas evidências.

Origem: EQUIPE-12. Verificação: CT-019. Método: Instalação e demonstração.

3.3 Catálogo formal de regras de negócio

RN-001 Arquitetura discreta

Somente as combinações de palavra, registradores e memória descritas em RF-001 são válidas. A memória é endereçada em palavras, começando no endereço 0.

Origem: decisão da equipe sobre o modelo educacional. Verificação: CT-001.

RN-002 Aritmética modular

Registradores e células de memória armazenam inteiros sem sinal de 0 a 2^bits - 1. ADD, SUB e ADDI produzem o resultado módulo 2^bits; não há exceção por overflow aritmético.

Origem: decisão da equipe sobre o modelo educacional. Verificação: CT-020.

RN-003 ISA própria

O conjunto válido é MOV, ADD, SUB, AND, OR, XOR, ADDI, LOAD, STORE, JMP, JZ, JNZ e HALT. Não é uma implementação RISC-V nem um formato binário de instruções de uma CPU comercial.

Origem: decisão da equipe sobre o modelo educacional. Verificação: CT-003.

RN-004 Operandos

MOV aceita destino registrador e origem imediata ou registrador. ADD, SUB, AND, OR e XOR usam três registradores; ADDI usa destino, origem registrador e imediato. LOAD e STORE usam registrador e endereço direto de memória.

Origem: decisão da equipe sobre o modelo educacional. Verificação: CT-021.

RN-005 Imediatos e endereços

Os imediatos aceitos pertencem ao intervalo de -2^(bits-1) até 2^bits - 1 e são normalizados módulo 2^bits. Endereços devem pertencer à memória configurada; alvos de salto devem apontar para instruções existentes.

Origem: decisão da equipe sobre o modelo educacional. Verificação: CT-022.

3.3 Catálogo formal de regras de negócio

RN-006 Desvios condicionais

JZ desvia quando o registrador informado é zero; JNZ desvia quando é diferente de zero. JMP sempre desvia. O destino é um rótulo resolvido ou um índice de instrução começando em 0.

Origem: decisão da equipe sobre o modelo educacional. Verificação: CT-023.

RN-007 Parada explícita

HALT encerra a execução e conta como uma instrução executada. A tentativa de executar após a última instrução sem HALT deve produzir erro controlado; um laço só pode avançar até o limite do lote.

Origem: decisão da equipe sobre o modelo educacional. Verificação: CT-024.

RN-008 Contagem

Cada instrução executada com sucesso corresponde a um ciclo didático. Esses ciclos não representam frequência, latência física nem tempo de uma CPU real.

Origem: decisão da equipe sobre o modelo educacional. Verificação: CT-004.

RN-009 Projeto persistente

O arquivo de projeto versão 1 guarda somente metadados de formato, configuração e fonte. Abrir carrega a fonte no editor sem montá-la; Montar, Passo ou Executar inicia uma CPU para o projeto. Não há continuidade de uma execução anterior.

Origem: decisão da equipe sobre o modelo educacional. Verificação: CT-009.

RN-010 Limites de exportação HDL

A exportação Verilog representa somente a ALU combinacional. O produto não exporta uma CPU completa, pipeline, caches, RTL de controle, temporização, layout físico ou fluxo de fabricação.

Origem: decisão da equipe sobre o modelo educacional. Verificação: CT-011.

4 Critérios de verificação e validação

4.1 Métodos de verificação

Inspeção examina código, arquivos e rótulos. Análise confere a coerência da especificação e a derivação de regras. Demonstração reproduz o uso da interface. Teste executa entradas com oráculos determinados antes da execução. A tabela resume os casos; a evidência real deve indicar data, ambiente e resultado obtido.

Casos	Método	Objeto
CT-001 a CT-003	Teste	Configuração e montagem válidas e inválidas
CT-004 a CT-006	Teste e demonstração	Passo, limite de execução e reinício
CT-007	Demonstração	Código e memória simultâneos
CT-008 e CT-009	Teste	JSON restrito e persistência
CT-010 e CT-011	Teste e inspeção	CSV e módulo ALU
CT-012 a CT-014	Teste e medição	Soma, lote e ida e volta JSON
CT-015 a CT-019	Inspeção e teste	Interface, atomicidade e ambiente
CT-020 a CT-024	Teste	Módulo aritmético, operandos, limites e desvios

Critério geral de aceitação da etapa: disponibilizar código executável e documentação; reproduzir o programa de exemplo; rejeitar entradas inválidas sem alteração indevida; manter a relação entre requisito e teste. Casos manuais não executados devem continuar identificados como planejados. Testes automatizados executados constam do registro de evidências entregue com o projeto.

A SRS fornece um contrato de aceitação. O grupo auditor deve derivar sua própria suíte a partir deste contrato e do código, preservando a independência exigida no enunciado. Não há resultados de auditoria externa, cobertura ou mutação atribuídos a esta etapa.

4.1 Casos de aceitação reproduzíveis

CT-012 Soma de 1 a 5

Pré-condição: aplicação aberta com arquitetura 8 bits, 8 registradores e 64 palavras; soma.asm carregado. Passos: montar; executar com limite 1000; ler registradores, memória e estado. Entrada: programa fornecido nos exemplos. Resultado esperado: R0 = 0, R1 = R2 = memória[0] = 15, PC = 8, 20 ciclos e HALT. Pós-condição: rastro preservado para exportação.

CT-016 Rejeição atômica de entrada inválida

Pré-condição: projeto válido montado e um passo executado; registrar configuração, fonte, PC, ciclos, registradores e memória. Passos: tentar importar um projeto com versão 99; comparar o estado anterior e posterior. Entrada: JSON estruturalmente válido com versão não suportada. Resultado esperado: mensagem de erro; nenhum campo do projeto corrente substituído. Pós-condição: o projeto anterior continua utilizável.

CT-023 Desvios condicionais

Pré-condição: CPU reiniciada, banco com pelo menos 4 registradores. Passos: executar um programa com JZ sobre zero e outro com JNZ sobre valor 1; repetir com as condições invertidas. Entrada: destinos válidos e instruções que atribuem marcadores diferentes. Resultado esperado: destino tomado somente quando a condição definida em RN-006 for verdadeira. Pós-condição: rastro identifica a sequência efetivamente executada.

CT-013 Execução limitada e tempo

Pré-condição: registrar sistema operacional, processador, compilador e configuração 8/8/64; montar JMP 0. Passos: medir com relógio monotônico um lote de 1000 passos. Entrada: limite 1000. Resultado esperado: retorno com 1000 ciclos; meta temporal de até 2 segundos no ambiente identificado. Pós-condição: programa pode continuar em novo lote. O valor medido deve ser anotado, sem generalizar para todo computador.

5 Matriz de rastreabilidade

5.1 Relação bidirecional entre requisito e teste

A matriz completa está no arquivo MATRIZ_RASTREABILIDADE.csv. Cada linha contém origem, ID de requisito, componente e caso de aceitação. A consulta origem → requisito → teste permite verificar a cobertura da necessidade; a consulta teste → requisito → origem identifica a justificativa de uma verificação. Os mesmos identificadores aparecem no catálogo e nos casos.

Origem	Requisitos	Verificação
F1	RF-007; RNF-004	CT-007; CT-015
EQUIPE-01	RF-001; RN-001	CT-001
EQUIPE-02	RF-002; RF-003; RN-003 a RN-005	CT-002; CT-003; CT-021; CT-022
EQUIPE-03	RF-004 a RF-006; RN-002; RN-006 a RN-008	CT-004 a CT-006; CT-020; CT-023; CT-024
EQUIPE-05	RF-008; RF-009; RNF-003; RN-009	CT-008; CT-009; CT-014
EQUIPE-06 e 07	RF-010; RF-011; RN-010	CT-010; CT-011
EQUIPE-08 a 12	RNF-001; RNF-002; RNF-005 a RNF-008	CT-012; CT-013; CT-016 a CT-019

F2 não gera requisitos de implementação: sua função é fundamentar a contraprova de viabilidade. A1 gera obrigações acadêmicas, tratadas na composição da entrega e na organização da equipe. Esses vínculos não são confundidos com requisitos operacionais da CPU.

O estado inicial dos casos de aceitação na matriz é planejado. A coluna de testes automatizados informa vínculos com funções de tests/test_core.cpp, sem afirmar que elas substituem a execução integral dos casos manuais. Os resultados reais devem ser consultados no registro de evidências. A presença de um vínculo não equivale a porcentagem de cobertura.

6 Projeto e implementação da solução computacional

A arquitetura separa núcleo e interface. O núcleo contém validação da configuração, montagem, CPU simulada, leitura e gravação de projeto, exportação do rastro e geração de ALU. A interface Win32 chama esse núcleo, apresenta estados e encaminha mensagens de erro. Essa separação permite testar a lógica sem depender de cliques ou de uma sessão gráfica.

Componente	Responsabilidade
src/core.hpp	Modelo de CPU, ISA, validação e exportações
src/main.cpp	Janela, editor e painéis de estado
tests/	Testes automatizados do núcleo
samples/	Programas e projetos para reprodução
documentacao/	Relatório, SRS, matriz e roteiro
evidencias/	Registros reais da verificação realizada

O caminho principal é: escolher parâmetros; editar a fonte; montar; executar um passo ou um lote; conferir registradores e memória; exportar resultados. O PC referencia instruções montadas, não bytes da memória de dados. Os registradores e a memória não são editáveis por fora da execução; o usuário manipula a arquitetura e o programa para produzir mudanças de estado.

A montagem percorre a fonte e consulta um mapa ordenado de rótulos, com limite superior O(n log n) para n caracteres. Um lote de k passos tem custo O(k), com k no máximo 1000 e estados de tamanho limitado pela arquitetura. A interface percorre esses estados ao atualizar a exibição. Essa análise descreve o processamento e não constitui medição de desempenho.

O módulo Verilog produzido tem natureza combinacional e sem estado. A correção textual do gerador não comprova, sozinha, síntese em uma ferramenta específica, correção temporal ou integração em FPGA. Essas atividades são extensões possíveis e não critérios de conclusão da etapa 1.

7 Organização e transferência do projeto

A matriz a seguir é uma proposta de alocação para validação pelo Grupo 2. Ela não afirma que cada integrante já executou as atividades. Os papéis cobrem desenvolvimento, análise, garantia de qualidade e documentação. A colaboração entre papéis é esperada, mantendo independência na auditoria do outro grupo.

Integrante	Papel proposto	Etapa 1	Etapa 2
Alexandre	Coordenação e arquitetura	Consolidar escopo e entrega	Coordenar integração e revisão final
Eliel	Análise de requisitos	Revisar RF e origens	Revisar testes derivados da SRS
Leonardo	Desenvolvimento do núcleo	Revisar montador e ISA	Apoiar análise de fluxo e correções
Luan	Desenvolvimento da interface	Revisar operação e mensagens	Reproduzir falhas de interação
Maitê	Documentação	Revisar relatório e referências	Consolidar relatório de auditoria
Marcos	Qualidade e testes funcionais	Revisar limites e oráculos	Projetar classes e valores limite
Whanderlon	Qualidade estrutural	Revisar testes internos	Analisar GFC, cobertura e mutação
Willian	Integração e distribuição	Revisar pacote e demonstração	Manter ambiente e evidências

Em 27 de outubro de 2026, o Grupo 2 deve apresentar o sistema e formalizar a entrega do código e de toda a documentação ao Grupo 1 e à professora. O Grupo 1 audita o ChipLab; o Grupo 2 audita o sistema recebido do Grupo 1. Nenhuma mensagem ou submissão externa foi realizada por este pacote.

O pacote deve conter instruções de execução, fontes, exemplos e testes, além deste relatório. Na transferência, o grupo deve registrar data, versão entregue e responsável pelo recebimento. Até 16 de dezembro de 2026, deve conservar a versão entregue e as dependências necessárias, registrar correções e manter uma forma reproduzível de executar o produto.

8 Considerações finais da primeira etapa

A solução escolhida transforma uma necessidade pública de observação de código e memória em um sistema desktop relacionado à arquitetura de processadores. O escopo mantém regras suficientes para análise de decisões e validações: parâmetros discretos, limites de operandos, aritmética modular, saltos condicionais, parada explícita e persistência restrita.

A verificação interna registrou 9 grupos de testes do núcleo, com 104 verificações aprovadas, e 18 verificações de integração com controles Win32 reais em janela oculta. Os cenários incluem soma de 20 ciclos, rastro CSV, limites de passos e fonte, rejeição de R99 e reinicialização. Capturas e logs estão em evidencias; esses resultados não representam auditoria externa ou porcentagem de cobertura.

Um lote de 1000 saltos, com temporizador Win32, registrou 1339,74 ms em um ensaio, abaixo da meta de 2 segundos. O ambiente foi Windows NT 10.0.26200.0 x64, processador identificado como AMD64 Family 25 Model 33 Stepping 2 AuthenticAMD e GCC 13.2.0. A medição descreve essa execução em janela oculta e não caracteriza todos os computadores nem todos os cenários de uso.

Atividades reservadas à segunda etapa

O Grupo 2 deverá analisar o sistema recebido do Grupo 1 e projetar sua suíte independente. O relatório posterior deve incluir oráculos, casos formais, classes de equivalência, valores limite e tabelas de decisão, além dos registros de defeitos. A análise estrutural e os testes de mutação devem produzir métricas com método e ferramenta identificados.

Cobertura de caminhos, arestas e nós, complexidade ciclomática, escore de mutação e distribuição de defeitos não são preenchidos com valores estimados. O relatório unificado deve distinguir testes internos do desenvolvedor e auditoria externa e apresentar somente resultados obtidos e comprovados. A data prevista da segunda entrega é 24 de novembro de 2026.

Verificações antes da apresentação

A equipe deve completar os nomes institucionais se necessário, confirmar a distribuição de responsabilidades, executar o projeto no computador de apresentação e ensaiar o roteiro. A documentação explicita as decisões técnicas para permitir que esse preparo ocorra sem depender de explicações informais do autor do código.

Referências

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 10520: informação e documentação citações em documentos apresentação. Rio de Janeiro: ABNT, 2023. Referência de formatação indicada no enunciado.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação trabalhos acadêmicos apresentação. Rio de Janeiro: ABNT, 2024, versão corrigida 2025. Referência de formatação indicada no enunciado.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: informação e documentação referências elaboração. Rio de Janeiro: ABNT, 2025. Referência de formatação indicada no enunciado.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6027: informação e documentação sumário apresentação. Rio de Janeiro: ABNT, 2012. Referência indicada no enunciado.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6028: informação e documentação resumo resenha e recensão apresentação. Rio de Janeiro: ABNT, 2021. Referência indicada no enunciado.

CAL, Cristina Weber. Av3 Trabalho integrador. Disciplina Qualidade de Produto de Software. Unijorge, 2026. Documento da atividade fornecido pelo estudante. 10 p.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION. ISO/IEC 25010:2011: systems and software engineering systems and software quality requirements and evaluation (SQuaRE) system and software quality models. Geneva: ISO, 2011. Edição indicada na atividade.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION; INSTITUTE OF ELECTRICAL AND ELECTRONICS ENGINEERS. ISO/IEC/IEEE 29148:2018: systems and software engineering life cycle processes requirements engineering. Geneva: ISO, 2018. Estrutura indicada na atividade.

Referências das fontes públicas coletadas

QUANTRPETER. looking for cooperate on verilog converter. Logisim evolution, issue 1616, 31 dez. 2022. Disponível em: https://github.com/logisim-evolution/logisim-evolution/issues/1616. Acesso em: 26 out. 2026.

TROMBONEHERO. Feature request view memory and code disassembly at the same time. Ripes, issue 408, 2 jun. 2025. Disponível em: https://github.com/mortbopet/Ripes/issues/408. Acesso em: 26 out. 2026.

Anexo A Registro dos recortes coletados

F1 e F2 foram consultadas como fontes públicas de requisitos não estruturados. Os trechos literais selecionados aparecem nas páginas de levantamento e são preservados nos arquivos de fontes incluídos na documentação. A descrição da necessidade e a tradução explicativa são interpretações do Grupo 2; não representam uma aprovação dos autores das solicitações.

A escolha do recorte responde à finalidade de documentar a evidência necessária para a decisão sem reproduzir integralmente discussões de terceiros. Para leitura do contexto completo, o auditor deve acessar as URLs originais. O texto bruto anexado é o recorte efetivamente coletado, acompanhado de autoria, data e endereço.

Apêndice A Conjunto de arquivos da entrega

SRS.md contém a especificação canônica. MATRIZ_RASTREABILIDADE.csv contém os vínculos completos. Roteiro_Apresentacao.md organiza a exposição e a demonstração. O código fonte, os testes, os exemplos e os registros de evidências integram a mesma distribuição. Os arquivos deste apêndice são produzidos para o projeto e podem ser revistos pela equipe.

Versão documental: 1.0, preparada em 26 de outubro de 2026 para a etapa com entrega em 27 de outubro de 2026. Identificação: Unijorge, Engenharia da Computação, Salvador, Grupo 2.